



Image Compression – An Introduction

CSE 4883: Digital Image Processing – Finals Week 5

[Lec Raiyan Rahman](#)

Dept of CSE, UIU

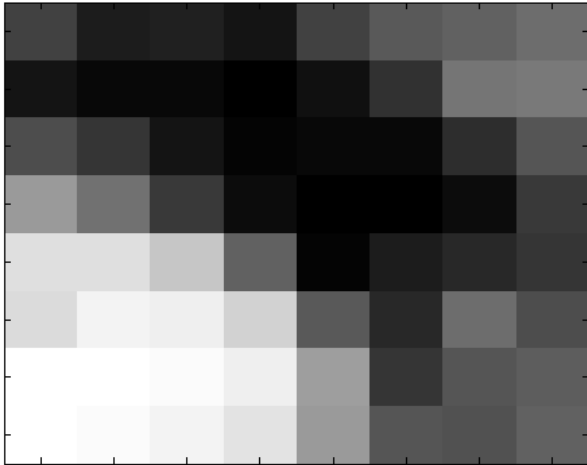
raiyan@cse.uiu.ac.bd

- Data are the means by which information is conveyed.
- Various amounts of data can be used to represent the same amount of information, representations that contain irrelevant or repeated information are said to contain redundant data.
- Information is an interpretation of the data in a meaningful way. Information is an elusive concept; it can be application specific.
- Data **not equal to** Information.
- For digital images, data refers to the pixel gray level values that correspond to the brightness of a pixel at a point in space.
- The term data compression refers to the process of reducing the amount of data required to represent a given quantity of information.

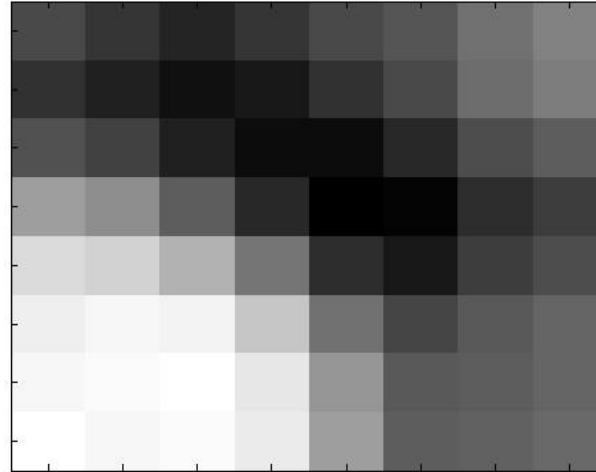
- ✓ The field of image compression continues to grow at a rapid pace.
 - ✓ Image compression is a method that reduces the amount of memory it takes to store in image, while retaining necessary information.
 - ✓ Information retention depends on the applications.
 - ✓ As we look to the future, the need to store and transmit images will only continue to increase faster than the available capability to process all the data.
-
- ✓ Applications that require image compression are many and varied such as:
 1. Internet,
 2. Businesses,
 3. Multimedia,
 4. Satellite imaging,
 5. Medical imaging

Can You Tell the Difference?

Original



Compressed



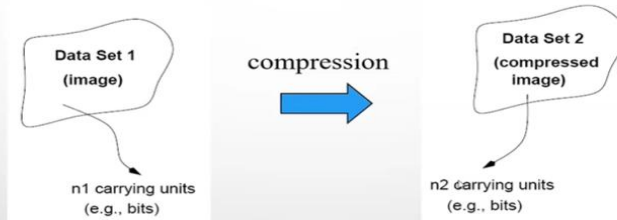
Original



Compressed



COMPRESSION RATIO



Let n_1 and n_2 denote the number of bits in two representations of the same information

COMPRESSION RATIO

RELATIVE DATA REDUNDANCY OF THE REPRESENTATION WITH n_1 BITS

$$R = 1 - \frac{1}{C}$$

COMPRESSION RATIO

$$C = \frac{n_1}{n_2}$$

If $C = 10$ (sometimes written 10:1), for instance, the larger representation has 10 bits of data for every 1 bit of data in the smaller representation. The corresponding relative data redundancy of the larger representation is 0.9 ($R = 0.9$), indicating that 90% of its data is redundant.

- ✓ The reduced file created by the compression process is called the *compressed file* and is used to reconstruct the image, resulting in the *decompressed image*
- ✓ The original image, before any compression is performed, is called the *uncompressed image file*
- ✓ The ratio of the original, uncompressed image file and the compressed file is referred to as the *compression ratio*

The compression ratio is denoted by:

$$\text{Compression Ratio} = \frac{\text{Uncompressed file size}}{\text{Compressed file size}} = \frac{SIZE_U}{SIZE_C}; \text{Often written as } \rightarrow SIZE_U : SIZE_C$$

EXAMPLE 10.1.1: The original image is 256x256 pixels, single-band (grayscale), 8-bits per pixel. This file is 65,536 bytes (64k). After compression the image file is 6,554 bytes. The compression ratio is: $SIZE_U/SIZE_C = 65536/6554 = 9.999 \approx 10$. This can also be written as 10:1

This is called a "10 to 1 compression", a "10 times compression", or can be stated as

"compressing the image to 1/10 its original size". Another way to state the compression is to use

the terminology of *bits per pixel*. For an NxN image:

$$\text{Bits per pixel} = \frac{\text{Number of bits}}{\text{Number of pixels}} = \frac{(8)(\text{Number of bytes})}{N \times N}$$

EXAMPLE 10.1.2: Using the preceding example, with a compression ratio of $65,536/6,554$ bytes, we want to express this as bits per pixel. This is done by first finding the number of pixels in the image: $256 \times 256 = 65,536$ pixels. We then find the number of bits in the compressed image file: $(6,554 \text{ bytes})(8 \text{ bits/byte}) = 52,432$ bits. Now we can find the bits per pixel by taking the ratio: $52,432/65,536 = 0.8 \text{ bits/pixel}$

- ✓ The reduction in file size is necessary to meet the bandwidth requirements for many transmission systems, and for the storage requirements in computer databases
- ✓ Also, the amount of data required for digital images is enormous

Image Compression

- ✓ There are two primary types of image compression methods:
 1. *Lossless compression methods:*
 - Allows for the exact recreation of the original image data, and can compress complex images to a maximum 1/2 to 1/3 the original size – 2:1 to 3:1 compression ratios.
 - Preserves the data *exactly*.
 2. *Lossy compression methods:*
 - Data loss, original image cannot be re-created exactly
 - Can compress complex images 10:1 to 50:1 and retain high quality, and 100 to 200 times for lower quality, but acceptable, images

TYPES OF DATA REDUNDANCY

- ❑ Three principal types of data redundancies that can be identified and exploited in digital images
 1. Coding redundancy
 2. Spatial or temporal (interpixel) redundancy
 3. Psychovisual redundancy (irrelevant information)

- ❑ Data compression attempts to reduce one or more of these redundancy types.

- **Coding redundancy:** A code is a system of symbols (letters, numbers, bits, and the like) used to represent a body of information or set of events. Each piece of information or event is assigned a sequence of code symbols, called a code word. The number of symbols in each code word is its length. The 8-bit codes that are used to represent the intensities in most 2-D intensity arrays contain more bits than are needed to represent the intensities
- **Spatial and temporal redundancy:** Because the pixels of most 2-D intensity arrays are correlated spatially (i.e., each pixel is similar to or dependent upon neighboring pixels), information is unnecessarily replicated in the representations of the correlated pixels. In a video sequence, temporally correlated pixels (i.e., those similar to or dependent upon pixels in nearby frames) also duplicate information.
- **Irrelevant information:** Most 2-D intensity arrays contain information that is ignored by the human visual system and/or extraneous to the intended use of the image. It is redundant in the sense that it is not used.



r_k	$p_r(r_k)$	Code 1	$l_1(r_k)$	Code 2	$l_2(r_k)$
$r_{87} = 87$	0.25	01010111	8	01	2
$r_{128} = 128$	0.47	01010111	8	1	1
$r_{186} = 186$	0.25	01010111	8	000	3
$r_{255} = 255$	0.03	01010111	8	001	3
r_k for $k = 87, 128, 186, 255$	0	—	8	—	0

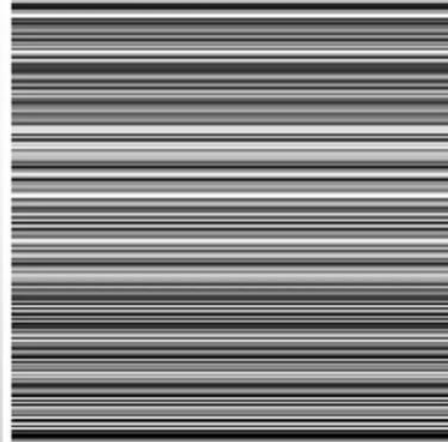
SPATIAL AND TEMPORAL REDUNDANCY

CONSIDER THE FOLLOWING IMAGE OF SIZE 256×256 .
IN THE CORRESPONDING 2-D IMAGE:

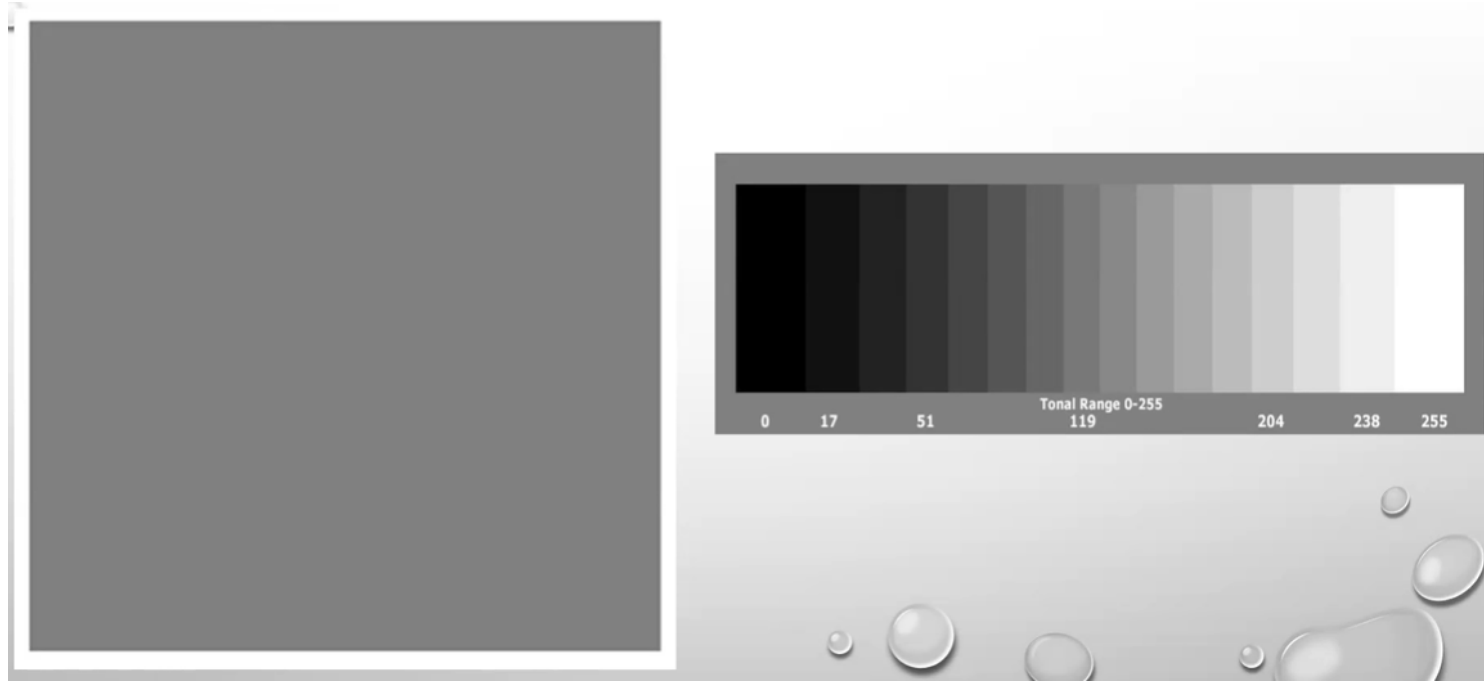
256, 255

256, 0

256, 1



Irrelevant information



- ❑ $r_k \rightarrow$ Input Intensity Value e.g. 0 – 255 for grayscale image
- ❑ $l(r_k) \rightarrow$ No. of bits used to represent r_k
- ❑ Then average no of bits required to represent each pixel is

$$L_{avg} = \sum_{k=0}^{L-1} l(r_k) p_r(r_k)$$

Say, uniform width str. (all pixels are using 8 bits to represent intensity)
 $\rightarrow$ equal length code (fixed length code)
 $\rightarrow$ m-bit fixed length code (8 bit fixed length code in our case)

- ❑ In this case, $l(r_k) = 8$ [each rq. 8 bits in fixed length]

$$L_{avg} = 8 \sum_{k=0}^{L-1} p_r(r_k) = 8 * 1 = 8$$

- ❑ Coding redundancy tries to reduce L_{avg}
- ❑ Thus, total no. of bits required to represent an $M \times N$ image is MNL_{avg}
- ❑ The code can be an equal length code, or variable length code.

- ❑ For code 1, $L_{avg} = 8$
- ❑ On the other hand, using code 2, the average length of the encoded pixels is

$$L_{avg} = 0.25(2) + 0.47(1) + 0.25(3) + 0.03(3) = 1.81 \text{ bits}$$

- ❑ The resulting compression and corresponding relative redundancy are

Thus, 77.4% of the data in the original 8-bit 2-D intensity array is redundant.

$$C = \frac{256 \times 256 \times 8}{256 \times 256 \times 1.81} \approx 4.42$$

$$R = 1 - \frac{1}{4.42} = 0.774$$

- Suppose we have a simple 3x3 grayscale image like this:

50 50 50

50 50 50

50 50 50

In this example, all pixel values are the same (50), which means they can be represented using a single value. However, due to the 8-bit depth, each pixel is still using 8 bits, resulting in redundancy.

- **Mathematical Calculation:**

- Original Image Size = Width $\times$ Height $\times$ Bit Depth (bit required to represent 1 pixel)
- In this example, the original image size = $3 \times 3 \times 8 = 72$ bits
- However, we can represent the entire image using a single pixel value (50), which requires just 8 bits.
This reduction is due to the elimination of coding redundancy.

- **Reduced Image Size = Width $\times$ Height $\times$ Bit Depth**

- Reduced image size = $3 \times 3 \times 1 = 8$ bits

- **Coding Redundancy:**

- Coding Redundancy = Original Image Size - Reduced Image Size
- Coding redundancy = 72 bits - 8 bits = 64 bits

Given:

Original Image Size = 72 bits

Reduced Image Size = 8 bits

Compression Ratio:

Compression Ratio = Original Image Size / Reduced Image Size

Compression Ratio = 72 bits / 8 bits = 9

In this example, the coding redundancy is 64 bits, which means that we have 64 bits of unnecessary information due to the uniform pixel values. Compression techniques aim to reduce or eliminate this redundancy to make image storage and transmission more efficient.

It's important to note that this example is a simple illustration. In real-world scenarios, more sophisticated compression algorithms like **Huffman coding**, Run-Length Encoding (RLE), and Discrete Cosine Transform (DCT) are used to reduce coding redundancy in images. These techniques analyze patterns and redundancies in the image data to achieve better compression ratios.

- A measure to reduce coding redundancy
- Most popular coding redundancy technique
- Variable length code
- Min length code is assigned to one with highest probability

What if instead of a minimum fixed length for each pixel, we could have a variable length? Could we reduce the overall bit length of an image even further?

Yes! That's exactly what Huffman coding does.

Huffman Coding for Image Compression:

Huffman coding is a lossless compression technique that reduces redundancy in the representation of symbols (in this case, pixel values in an image).

It assigns shorter codes to more frequently occurring symbols and longer codes to less frequently occurring symbols. This way, the most common symbols are represented with fewer bits, resulting in efficient compression.

1. **Frequency Analysis:** Calculate the frequency of each symbol (pixel intensity) in the image. The more frequent a symbol is, the shorter its code will be.
2. **Priority Queue (Min Heap):** Create a priority queue (min heap) based on the calculated frequencies. Each node in the heap will represent a symbol and its frequency.
3. **Building Huffman Tree:** Continuously extract two nodes with the lowest frequencies from the heap, combine them into a new node (with a frequency equal to the sum of their frequencies), and insert the new node back into the heap. Repeat this step until only one node (the root of the Huffman tree) remains in the heap.

4. **Assigning Codes:** Traverse the Huffman tree from the root to each leaf. Assign a '0' for each left branch and '1' for each right branch. The path from the root to a leaf represents the code for that symbol.
5. **Generate Huffman Table:** Create a table that maps each symbol to its corresponding Huffman code. This table is used for encoding and decoding.

An Example Simulation (Follow Classnote)



Thank You